

Resumen teórico — Sistemas Operativos

Notas del curso 7: *Planificación de uso de CPU*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico del capítulo 7 de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC), dedicado a la planificación de uso de CPU. Se presentan las funciones del scheduler, los criterios clásicos de planificación y los algoritmos que los implementan: *First-Come First-Served*, *Shortest Job First* (con su variante preemptiva *SRTF*), *Round Robin*, planificación por prioridades estáticas y dinámicas, *Multilevel Queues* y su variante con feedback (MLF). Se discuten las propiedades de cada uno (preemption, fairness, starvation, contención), las métricas de evaluación (turnaround, espera, respuesta, throughput) y se incluye el caso de estudio del scheduler de Linux. La última parte trata la planificación en multiprocesadores: arquitecturas AMP y SMP, multi-core, hyperthreading y la tensión entre *processor affinity* y *load balancing*. El objetivo es servir como material de estudio para preparar el Práctico 4 y el segundo parcial.

Índice

1. Introducción	2
2. Tipos de schedulers	2
2.1. Long-term scheduler	2
2.2. Short-term scheduler	2
3. Criterios de planificación	3
3.1. Métricas en una corrida concreta	3
4. Preemption: apropiativo vs. no apropiativo	3
5. Algoritmos de planificación	4
5.1. First-Come First-Served (FCFS)	4
5.2. Shortest Job First (SJF) y Shortest Remaining Time First (SRTF)	5
5.3. Round Robin (RR)	6
5.4. Planificación por prioridades	6
5.5. Multilevel Queues (MLQ)	7
5.6. Multilevel Feedback (MLF)	8
6. Caso de estudio: el scheduler de Linux	8
6.1. Representación de procesos y threads	8
6.2. Rango de prioridades	8
6.3. El valor <code>nice</code>	9
6.4. Algoritmos a lo largo del tiempo	9
7. Multiprocesadores	9
7.1. AMP vs. SMP	9
7.2. Load balancing	9
7.3. Processor affinity	10

7.4. Multi-core e hyperthreading	10
8. Comparación rápida de algoritmos	11
9. Resumen final	11

1. Introducción

Una computadora moderna corre decenas de procesos a la vez con un número limitado de CPUs. Para que todos avancen, alguien tiene que decidir, instante a instante, *quién* usa la CPU. Esa decisión es trabajo del **scheduler** o *planificador*. Las Notas del curso (capítulo 7) lo definen así:

Definición

El **scheduler de CPU** de un sistema operativo es el encargado de determinar:

- *cuándo* una tarea debe continuar con su ejecución (asignarle una CPU),
- *dónde* (en qué procesador la ejecuta), y
- *por cuánto tiempo*.

Estas tres preguntas no se contestan igual en todos los sistemas. En un *batch system* de los años 70 importa terminar la mayor cantidad de *jobs* por hora; en una computadora interactiva moderna importa que la respuesta del editor de texto sea instantánea. Por eso existen distintos algoritmos: cada uno optimiza un criterio distinto, y el rol del diseñador del SO es elegir el que mejor sirve a la carga de trabajo esperada.

2. Tipos de schedulers

Las Notas distinguen dos clases de planificadores que operan en escalas de tiempo distintas:

2.1. Long-term scheduler

Aparece en *batch systems* y en clusters/HPC. Los usuarios *submits jobs* a una cola (*jobs pool*), donde cada job declara los recursos que necesita (número de CPUs, memoria, tiempo estimado de ejecución). El long-term scheduler elige qué jobs se admiten al sistema y en qué orden, con el objetivo típico de *minimizar el tiempo medio de espera*. El algoritmo **Shortest Job First** (SJF) es el ejemplo paradigmático: si los tiempos de ejecución se conocen, ejecutar primero los más cortos minimiza la espera promedio.

2.2. Short-term scheduler

Es el que usan los SO modernos *time-sharing*. Decide en cada momento, entre los procesos en estado **READY**, cuál recibe la CPU a continuación. Trabaja en escalas de milisegundos o microsegundos. Es el scheduler que se estudia en este capítulo y el que aparece en todos los ejercicios del Práctico 4.

Idea intuitiva

La diferencia clave: el long-term scheduler decide *si* un job entra al sistema (a veces queda esperando horas en la cola); el short-term scheduler decide qué proceso *ya admitido* corre en la CPU *ahora*.

3. Criterios de planificación

¿Qué significa que un scheduler sea “bueno”? Las Notas listan los criterios habituales:

- **Maximizar la utilización de la CPU.** Que la CPU esté ocupada haciendo trabajo útil el mayor porcentaje de tiempo posible. Lo ideal es 100 %; en la práctica, 40–90 % según la carga.
- **Justicia (*fairness*).** Todas las tareas deberían recibir tiempos parejos de uso de CPU.
- **Throughput (rendimiento).** Número de procesos completados por unidad de tiempo.
- **Tiempo de retorno (*turnaround*).** Tiempo total que un proceso pasa en el sistema, desde que arribó hasta que terminó. Suma los tiempos en estados **READY**, **RUNNING** y **WAITING**.
- **Tiempo de espera.** Tiempo que el proceso pasó en **READY** esperando por la CPU.
- **Tiempo de respuesta.** Latencia desde que llega un requerimiento hasta que el sistema empieza a procesarlo. Crítico en sistemas interactivos.

Adicionalmente, un scheduler debe ser **eficiente** (mínima sobrecarga: tomar la decisión rápido, ya que se ejecuta muchas veces por segundo) y **general** (funcionar razonablemente en cualquier arquitectura para distintas cargas de trabajo).

Importante

Estos criterios **se contraponen entre sí**. Maximizar throughput puede afectar negativamente al tiempo de respuesta; favorecer fairness puede empeorar el throughput total. Por eso no existe “el” mejor algoritmo: cada uno hace bien una métrica a costa de otras.

3.1. Métricas en una corrida concreta

Las fórmulas que se usan en los ejercicios:

$$\begin{aligned}\text{Espera}_i &= \text{Inicio}_i - \text{Arribo}_i \quad (\text{algoritmos no preemptivos sin I/O}) \\ \text{Turnaround}_i &= \text{Fin}_i - \text{Arribo}_i \\ \text{Respuesta}_i &= \text{PrimerInicio}_i - \text{Arribo}_i\end{aligned}$$

En la versión preemptiva con I/O, $\text{Espera}_i = \text{Turnaround}_i - \text{CPU}_i - \text{WAIT}_i$.

Los promedios se calculan dividiendo por la cantidad de procesos: $\overline{W} = \frac{1}{N} \sum_i W_i$ y análogamente para $\overline{T}$ y $\overline{R}$.

4. Preemption: apropiativo vs. no apropiativo

Una propiedad transversal a todos los algoritmos es si pueden o no quitarle la CPU a un proceso en plena ejecución.

Definición

Un algoritmo de planificación es **preemptivo** (apropiativo) si el scheduler puede desalojar a un proceso que está corriendo, antes de que termine su ráfaga de CPU. Es **no preemptivo** si, una vez que un proceso recibe la CPU, la conserva hasta terminar la ráfaga (o bloquearse voluntariamente en una syscall).

La preemption se implementa con la *interrupción del timer*: el hardware genera ticks periódicos, el handler del kernel decide si el quantum se acabó y, si es así, invoca `yield()` para hacer

un context switch. Esto se conoce como *preemptive multitasking* y es la base de los sistemas time-sharing modernos.

Una alternativa histórica es el **cooperative multitasking**: los procesos liberan la CPU *voluntariamente* mediante una syscall. Si un proceso se “cuelga” o no coopera, monopoliza la CPU. Por eso los SO modernos usan preemption por timer.

5. Algoritmos de planificación

A partir de aquí se asume que el scheduler decide entre los procesos **READY** (también llamados **RUNNABLE**), es decir, los que ya están en condiciones de usar la CPU.

5.1. First-Come First-Served (FCFS)

Definición

FCFS (*primero que llega, primero servido*, también llamado FIFO) es el algoritmo más simple. Se mantiene una cola FIFO de **READY**. Cuando un proceso pasa a **READY** se encola al final; el primero de la cola toma la CPU. Es **no preemptivo**: el proceso conserva la CPU hasta terminar su ráfaga.

Ejemplo. Supongamos los procesos

Proceso	Arribo	Ráfaga
P1	0	8
P2	0.5	3
P3	3	5

El orden de ejecución es P1, P2, P3 (orden de arribo). Diagrama de Gantt:



Esperas: $W_{P1} = 0$, $W_{P2} = 8 - 0.5 = 7.5$, $W_{P3} = 11 - 3 = 8$. Promedio $\overline{W} = 5.17$.

Ventajas.

- Simplísimo: la cola FIFO basta para implementarlo.
- No requiere información sobre las ráfagas.
- No tiene problemas de *starvation* (todos avanzan en el orden de llegada).

Desventajas.

- Suele producir **tiempos de espera promedio grandes**.
- Sufre el **convoy effect**: si llega primero un proceso muy largo, arrastra la espera de todos los que llegan después.
- Inutilizable en sistemas time-sharing, porque al ser no preemptivo, un proceso CPU-bound puede acaparar la CPU durante mucho tiempo.

5.2. Shortest Job First (SJF) y Shortest Remaining Time First (SRTF)

Definición

SJF: el scheduler elige siempre el proceso con la *ráfaga más corta* entre los **READY**. La versión clásica es **no preemptiva** (típica de batch systems). La versión preemptiva, llamada **SRTF** (Shortest Remaining Time First), sí desaloja al proceso en ejecución si llega uno con ráfaga restante menor.

SJF se usa en *long-term schedulers* y en batch systems donde el usuario declara el tiempo estimado del job. Aplicado como *short-term scheduler*, lo correcto es llamarlo *shortest CPU-burst first*, porque lo que se considera no es la duración total del proceso sino la duración de su *próxima ráfaga de CPU* (el intervalo entre dos bloqueos por I/O).

Ráfaga. Una *ráfaga (burst)* de CPU es el tiempo que un proceso usa la CPU desde que es planificado hasta que la libera (por bloqueo en I/O o por terminar). El comportamiento típico de un proceso alterna ráfagas de CPU y períodos de WAIT:

```
burst1 sleeping burst2 sleeping ...
      syscall wakeup syscall
time -->
```

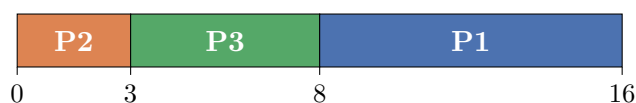
Estimación de la próxima ráfaga. Como SJF requiere conocer de antemano la duración, en un short-term scheduler hay que *estimarla*. La técnica clásica es el **promedio exponencial**: si t_n es la duración real de la última ráfaga y τ_n la última estimación, entonces

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n, \quad 0 \leq \alpha \leq 1.$$

α controla el peso relativo entre la *historia reciente* y la *historia pasada*. Casos extremos:

- $\alpha = 0$: $\tau_{n+1} = \tau_n$. La historia reciente no se tiene en cuenta.
- $\alpha = 1$: $\tau_{n+1} = t_n$. Solo importa la última ráfaga.
- $\alpha = 0.5$ es un balance común entre las dos.

Ejemplo. Mismos procesos del FCFS pero asumiendo que *todos arriban en $t = 0$* (caso clásico para mostrar la optimalidad de SJF). Orden por ráfaga: P2 (3), P3 (5), P1 (8).



Promedio de espera: $(0 + 3 + 8)/3 \approx 3.67$. Bastante mejor que los 5.17 de FCFS sobre los mismos procesos.

Ventajas.

- Es **óptimo en tiempo de espera promedio** cuando se conocen las ráfagas: ningún otro algoritmo logra menos espera promedio sobre el mismo conjunto.
- Reduce el *convoy effect*: un proceso largo no arrastra a los cortos.

Desventajas.

- **No se conoce** la duración de la próxima ráfaga (hay que estimarla).
- Puede producir **starvation**: si llegan continuamente procesos cortos, los largos pueden no ejecutarse nunca.
- En la versión preemptiva (SRTF), cada llegada implica una evaluación; la sobrecarga aumenta.

5.3. Round Robin (RR)

Definición

Round Robin es la variante preemptiva de FCFS, pensada para sistemas time-sharing. Se mantiene una cola FIFO de **READY**; al otorgarle la CPU a un proceso se le asigna un *quantum* (también llamado *time slice*). Si el proceso consume todo el quantum sin liberar la CPU, el timer lo desaloja, vuelve a **READY** y se encola al final. Si la libera antes (por I/O o por terminar), el quantum no le alcanza pero igual cede la CPU al siguiente.

Quantum. Es el parámetro central de RR. Hay un compromiso clásico:

- Quantum **muy grande**: RR converge a FCFS.
- Quantum **muy chico**: el sistema gasta mucho tiempo en cambios de contexto en lugar de hacer trabajo útil. En el límite, aproxima un *processor sharing* ideal pero con overhead inaceptable.

La elección habitual está entre 10 y 100 ms, y depende del costo del context switch en la arquitectura.

Ejemplo. Mismos procesos que antes, con $q = 2$:



P2 (la ráfaga más corta) termina antes que en FCFS, pero P1 (la más larga) termina mucho más tarde porque ejecuta en cuatro porciones intercaladas.

Ventajas.

- **Acota el tiempo de respuesta**: ningún proceso tiene que esperar más que $(N - 1) \cdot q$ unidades antes de ejecutar, donde N es la cantidad de procesos en **READY**.
- **Justo**: todos los procesos progresan regularmente.
- Sin starvation.

Desventajas.

- **Mayor turnaround promedio** que FCFS o SJF cuando hay procesos largos, porque la ejecución se fragmenta.
- Costo de los *context switches*: cada quantum agotado produce uno.

5.4. Planificación por prioridades

Definición

A cada proceso se le asocia un **número de prioridad**; el scheduler elige siempre el de mayor prioridad. La relación de orden puede ser arbitraria: en algunos sistemas un menor valor implica mayor prioridad (Linux real-time, 0–99); en otros, lo contrario (Linux normal vía *nice*, –20 a +19).

Estática vs. dinámica.

- **Estática**: la prioridad se asigna en tiempo de creación y no cambia. Es lo que usa el Práctico 4 Ejercicio 4.
- **Dinámica**: el sistema recalcula la prioridad periódicamente según el comportamiento del proceso. Es la base del scheduler 4.3BSD y, conceptualmente, de MLF.

Preemptiva vs. no preemptiva. El algoritmo puede implementarse en cualquiera de las dos formas: preemptivo si llegar una tarea de mayor prioridad desaloja a la actual; no preemptivo si la decisión solo se toma cuando la actual termina o se bloquea.

Ejemplo de prioridad dinámica (4.3BSD). La fórmula clásica es

$$\text{priority} = \frac{\text{recent_cpu_usage}()}{2} + 60,$$

donde mayor valor numérico implica menor prioridad efectiva. Cada segundo se recalcula. Un proceso CPU-bound acumula `recent_cpu` y su prioridad numérica sube (es decir, baja en términos efectivos). Un proceso I/O-bound, que se bloquea seguido, mantiene `recent_cpu` chico y conserva alta prioridad. Resultado: el sistema favorece a los interactivos sin negarles servicio a los CPU-bound, porque la penalización solo refleja la historia reciente.

Starvation y aging. El gran problema de las prioridades es la **starvation**: un proceso de baja prioridad podría no ejecutarse nunca si llegan continuamente otros más prioritarios. Es, por construcción, un algoritmo *no justo*.

Importante

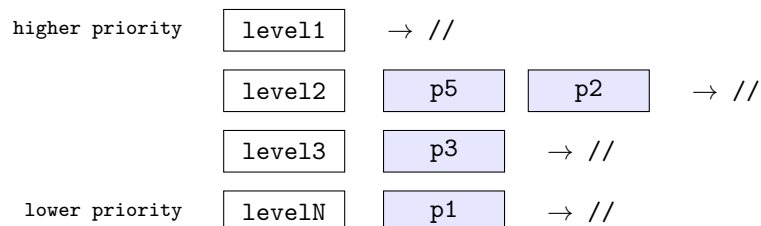
Aging (envejecimiento): incrementar gradualmente la prioridad de los procesos que lleven mucho tiempo esperando. Tarde o temprano todos llegan al frente de la cola, evitando starvation. Es la solución estándar al problema y aparece explícitamente en la teoría.

5.5. Multilevel Queues (MLQ)

Definición

En **Multilevel Queues** los procesos se clasifican en grupos. A cada grupo se le asigna una prioridad: es entonces un algoritmo basado en prioridades. El scheduler selecciona siempre el primer proceso del grupo de mayor prioridad. Cada nivel puede usar un algoritmo distinto internamente (típicamente RR).

Ejemplo común: dos grupos, “interactivos” (alta prioridad) y “background / CPU-bound” (baja prioridad). Los interactivos se priorizan para minimizar el tiempo de respuesta.



5.6. Multilevel Feedback (MLF)

Definición

MLF es la variante *dinámica* de MLQ: los procesos se **mueven entre niveles** según su comportamiento.

- Si un proceso **consume todo el quantum**, se considera CPU-bound y se *baja* un nivel.
- Si un proceso **no consume todo el quantum** (cede voluntariamente, típicamente por bloqueo en I/O), se considera interactivo y se *sube* de nivel (o se mantiene si ya está en el nivel más alto).

Cada nivel suele tener su propio quantum, típicamente menor en los niveles altos (más respuesta) y mayor en los bajos (menos overhead para los CPU-bound).

Idea intuitiva

La intuición: “los premios y los castigos son automáticos”. El scheduler aprende solo qué procesos son interactivos (los premia con prioridad alta) y cuáles CPU-bound (los relega a niveles bajos donde compiten entre ellos), sin que un humano tenga que clasificarlos a mano. Es lo más parecido a un scheduler “adaptativo” clásico.

MLF típicamente cumple el objetivo de *minimizar el tiempo de respuesta de los procesos interactivos sin negarle progreso a los CPU-bound*. La penalización a los CPU-bound es “compartirse el nivel inferior”, no quedarse afuera.

6. Caso de estudio: el scheduler de Linux

Las Notas dedican una sección al scheduler de Linux como ejemplo concreto.

6.1. Representación de procesos y threads

Cada proceso o thread se representa con la estructura `task_struct`. Un proceso multi-threaded se modela con varias instancias: una por el thread principal, una por cada thread hijo; todas comparten el mismo `pid`.

6.2. Rango de prioridades

Linux usa un scheduler basado en prioridades en el rango **0 a 139**, dividido en dos grandes clases:

- **Real-time (0–99)**. Para procesos del sistema y kernel threads que requieren garantías de tiempo. Políticas `SCHED_FIFO` (no preemptiva entre procesos del mismo nivel) o `SCHED_RR` (RR con quantum fijo).
- **Normal (100–139)**. Para procesos de usuario. Políticas `SCHED_NORMAL` (RR con quantum variable), `SCHED_BATCH` (procesos no interactivos) y `SCHED_IDLE` (los de menor prioridad).

Tareas de tiempo real. Una tarea *real-time* requiere que el sistema le asigne recursos en tiempos acotados (con vencimientos). Ejemplo clásico: una aplicación que graba un DVD necesita mantener el buffer de datos siempre lleno, porque el láser no puede parar de grabar. Si el SO se atrasa, la grabación falla.

6.3. El valor `nice`

Para procesos normales, la prioridad se determina a partir del valor **nice**, en el rango -20 (más prioritario) a $+19$ (menos prioritario):

$$\text{priority} = \text{nice} + 20.$$

La syscall `nice(value)` permite ajustarlo. Conceptualmente:

- Cualquier usuario puede *subir* su `nice` (bajar su prioridad): cederle prioridad a otros.
- Solo **root** puede *bajar* su `nice` (subir su prioridad), porque permitir que un usuario suba la suya rompería la equidad y abriría la puerta a denegación de servicio.

El valor de `nice` también determina el quantum asignado al proceso. Los procesos normales arrancan con prioridad 0.

6.4. Algoritmos a lo largo del tiempo

Internamente Linux fue cambiando su scheduler a lo largo de los años: **O(1) scheduler** en los 2000, **CFS** (*Completely Fair Scheduler*) entre 2007 y 2024, y desde 2024 el algoritmo **EEVDF** (*Earliest Eligible Virtual Deadline First*). El scheduling es **preemptivo**: si arriba una tarea con más alta prioridad que la corriente, ésta es desalojada y la nueva toma la CPU.

Las versiones recientes permiten integrar *schedulers a medida* mediante la *scheduling class* `sched_ext`, que admite implementar políticas como código eBPF cargado en el kernel.

7. Multiprocesadores

Hasta acá se asumió implícitamente que hay *una* CPU. En arquitecturas modernas hay varias: el scheduler tiene que decidir no solo *qué* ejecutar, sino *en qué procesador*.

7.1. AMP vs. SMP

Definición

AMP (Asymmetric Multiprocessing): una CPU *master* ejecuta el código del kernel y sus servicios; las demás (*workers*) ejecutan procesos de usuario. El scheduler corre en el master, lo que simplifica los problemas de paralelismo.

Definición

SMP (Symmetric Multiprocessing): todas las CPUs ejecutan tanto código de usuario como del kernel. Es el modelo de las arquitecturas modernas.

En SMP hay que poner especial cuidado en problemas de **conurrencia**: varias CPUs pueden estar ejecutando código del kernel en paralelo, así que las estructuras compartidas requieren spinlocks, instrucciones atómicas, barreras de memoria, etc.

7.2. Load balancing

Definición

Load balancing: emparejar la carga de trabajo entre todas las CPUs. Si una está sobrecargada y otra ociosa, el scheduler *migra* procesos para nivelar.

Esto requiere que el sistema pueda contabilizar la carga de cada CPU y mover procesos a las menos cargadas. En cada context switch un proceso puede continuar en otra CPU. En Linux

hay un *kernel thread* dedicado, llamado **migration**, que ejecuta **load_balance** para hacer este trabajo.

7.3. Processor affinity

La migración tiene un costo: **la caché de la CPU anterior queda invalidada** (o, mejor dicho, deja de servir) y la caché de la nueva tiene que volver a llenarse. Mientras se “calienta” la nueva caché, el proceso ejecuta más lento porque acumula *cache miss*.

Definición

Processor affinity: política del scheduler que trata de minimizar la migración de procesos entre CPUs, para preservar la inversión en caché.

Linux ofrece dos niveles:

- **Soft affinity** (default): el scheduler trata de no migrar pero puede hacerlo si compensa.
- **Hard affinity:** la syscall **sched_setaffinity** fija el proceso a un subconjunto específico de CPUs y el balanceador no puede sacarlo de ahí.

Importante

Affinity y load balancing se contraponen. Affinity pide no migrar; load balancing pide migrar para nivelar la carga. La solución es un compromiso: *migrar solo cuando el desbalance proyectado supera al costo esperado de la migración*. Esto se implementa con per-CPU runqueues, balanceadores periódicos y umbrales de desbalance.

7.4. Multi-core e hyperthreading

Multi-core. Una arquitectura multi-core replica varias CPUs (*cores*) en un mismo chip. Cada core puede usar múltiples *pipelines* para ejecutar varios threads en paralelo y minimizar los *memory stalls* (esperas por datos no cacheados).

Hyperthreading. Es la capacidad de un core de ejecutar más de un thread en paralelo, asignándoles pipelines distintos. Una CPU de dos cores con dos pipelines cada uno se ve como un multiprocesador de 4 CPUs lógicas.

Hart (RISC-V). En la arquitectura RISC-V, un *hart* es la abstracción de un *hardware thread* o *logical processor*: una unidad lógica de procesamiento con su propio conjunto de registros. Una implementación puede materializar varios harts en un mismo core mediante múltiples pipelines.

8. Comparación rápida de algoritmos

Algoritmo	Preemption	Convención	Ventaja principal	Desventaja principal
FCFS	No	Cola FIFO	Simple, sin starvation	Convoy effect, no time-sharing
SJF	No	Menor ráfaga	Mínimo tiempo de espera promedio (óptimo)	Requiere conocer ráfagas; starvation
SRTF	Sí	Menor remanente	Igual que SJF, reactivo a llegadas	Más context switches; starvation
RR	Sí	Cola FIFO + quantum	Acota tiempo de respuesta, fair	Mayor turnaround si hay procesos largos
Prioridades	Cualq.	Mayor prioridad	Refleja política del sistema	Starvation; necesita aging
MLQ	Sí	Múltiples colas + prio. fija	Separa clases de tareas	Clasificación estática
MLF	Sí	Colas con prio. dinámica	Premia interactivos, castiga CPU-bound	Complejo de afinar

9. Resumen final

1. El **scheduler de CPU** decide cuándo, dónde y por cuánto tiempo cada tarea ejecuta. Hay schedulers de *long-term* (admisión) y *short-term* (selección instantánea).
2. Los **criterios** clásicos — utilización, fairness, throughput, turnaround, espera, respuesta — *se contraponen*. No existe el algoritmo perfecto; se elige según la carga de trabajo.
3. La **preemption** es la propiedad que permite al kernel recuperar la CPU sin la cooperación del proceso, gracias a las interrupciones del timer. Es la base de los sistemas time-sharing.
4. **FCFS** es simple pero sufre el *convoy effect*. **SJF** (no preemptivo) y **SRTF** (preemptivo) minimizan la espera promedio si se conocen las ráfagas, pero admiten starvation. La duración se estima con *promedio exponencial* $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.
5. **Round Robin** acota el tiempo de respuesta usando un *quantum*; es la elección natural para time-sharing. El quantum es un compromiso: muy grande $\rightarrow$ FCFS; muy chico $\rightarrow$ overhead de context switches.
6. **Prioridades** reflejan la importancia que el sistema asigna a cada proceso. Pueden ser *estáticas* o *dinámicas*; pueden ser *preemptivas* o no. Padecen starvation, mitigada con *aging*.
7. **MLF** combina prioridades dinámicas con RR por nivel: promueve a los interactivos, releva a los CPU-bound, y es lo más cercano a un scheduler “adaptativo” clásico.
8. En **Linux** la prioridad va de 0 a 139, con clases real-time (0–99) y normal (100–139). El valor **nice** (−20 a +19) controla la prioridad y el quantum de los procesos de usuario. Históricamente: O(1) $\rightarrow$ CFS $\rightarrow$ EEVDF (2024).
9. En **multiprocesadores SMP** el scheduler debe lograr *load balancing* (nivelar carga) y *processor affinity* (preservar caché). Estos dos objetivos se contraponen y se resuelven con un compromiso: migrar solo si el desbalance supera el costo esperado de la migración.

Importante

La idea unificadora: no hay un mejor algoritmo de planificación. Cada criterio que se quiere maximizar (throughput, respuesta, fairness, afinidad de caché) se contrapone con otros, y el rol del diseñador del SO — y del scheduler concreto que se elija — es navegar esos compromisos para una carga de trabajo particular. La planificación es, en última instancia, un ejercicio permanente de equilibrio entre objetivos contrapuestos.